

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)

Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа № 2.2
по дисциплине: Дискретная математика
по теме: Задачи выбора

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: ст. преподаватель
Бондаренко Т.В.

Белгород 2023

Цель работы: приобрести практические навыки в использовании алгоритмов порождения комбинаторных объектов при проектировании алгоритмов решения задач выбора.

Задания

1. Ознакомиться с задачей (см. варианты заданий).
2. Определить класс комбинаторных объектов, содержащих решение задачи (траекторию задачи).
3. Определить, что в задаче является функционалом и способ его вычисления.
4. Определить способ распознавания решения по значению функционала.
5. Реализовать алгоритм решения задачи.
6. Подготовить тестовые данные и решить задачу.

Вариант 18

1. Имеется n станков и n деталей. Каждую деталь можно обработать на любом станке, но время обработки детали на одном станке может отличаться от времени ее обработки на другом. Нужно разместить детали по станкам так, чтобы суммарное время работы было наименьшим
2. Необходимо расставить n деталей на n станков, соответственно задача сводится к полнопереборному алгоритму всех перестановок. Закрепим станки, первому месту будет соответствовать первый станок, второму – второй, и т.д. Пронумеруем подряд также детали, соответственно траекториями будут являться все перестановки чисел от 1 до n
3. Функционалом будет являться значение выражения, являющегося суммой соответствующих ячеек заданной матрицы. Пусть P – полученная нами перестановка, а A – заданная матрица, тогда можно будет вычислить функционал как сумму $A[i][P[i]]$, $i = 0, n - 1$
4. Выберем каким-либо образом первую траекторию и обозначим её функционал как $\min$, запомнив соответствующую перестановку $best_case$. Будет минимизировать функционал; если функционал соответствующей перестановки P будет меньше, чем $\min$, то мы запоминаем данную перестановку в $best_case$, а её функционал в $\min$. Таким образом, найдя все перестановки, мы получим решение.

5.

```
#include <stdio.h>
#include <locale.h>

#define N 3

void writeVector(int a[], int size) {
    for (int i = 0; i < size; i++) {
        printf(" Для станка № %i - деталь № %i \n", i + 1, a[i] + 1);
    }
    puts("\n");
}

void readMatrix(int a[][N], size_t size) {
    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++) {
            printf("Введите время станка № %i для детали № %i : ", i + 1, j +
1);
            scanf("%i", &a[i][j]);
            printf("\n");
        }
    }
}

int functional(int a[][N], size_t size, int b[]) {
    int time = 0;
    for (int i = 0; i < size; i++) {
        time += a[i][b[i]];
    }
    return time;
}

void assignment(int a[], int b[], size_t size2) {
    for (int i = 0; i < size2; i++) {
        a[i] = b[i];
    }
}

int notInArr(int *a, int m, int x) {
    int i = 0;
    while ((i < m) && (a[i] != x))
        i++;
    return i == m;
}

void Except(int* a, int m, int* b, int n, int* c, int* k) {
    *k = -1;
    for (int i = 0; i < m; i++) {
        if (notInArr(b, n, a[i])) {
            (*k)++;
            c[*k] = a[i];
        }
    }
    (*k)++;
}

int P[N];
void findBestCase(int a[][N], int B[], int b, int i, int min, int bc[]) {
    for (int x = B[0]; x <= B[b - 1]; x++) {
        P[i - 1] = x;
        if ((i == N)) {
            int temp = functional(a, N, P);
            if (temp < min) {
```

```

        assignment(bc, P, N);
    }
}
else if (i != N) {
    int temp1[N];
    int t1 = 0;
    int temp2[] = { x };
    Except(B, b, temp2, 1, temp1, &t1);
    findBestCase(a, temp1, t1, i + 1, min, bc);
}
}
}

int createRandom(int a[][N], int bc[]) {
    int min = 0;
    for (int i = 0; i < N; i++) {
        min += a[i][i];
        bc[i] = i;
    }
    return min;
}

int main() {
    setlocale(LC_ALL, "RUS");
    int a[N][N];

    readMatrix(a, N);

    int B[N];
    for (int i = 0; i < N; i++) {
        B[i] = i;
    }

    int b = N;
    int i = 1;
    int bc[N];

    int min = createRandom(a, bc);

    findBestCase(a, B, b, i, min, bc);

    puts("Лучший случай: ");
    writeVector(bc, N);

    return 0;
}

```

6.

Пусть $n = 3$, тогда задача выглядит так:

Имеется 3 станка и 3 детали. Необходимо разместить детали по станкам так, чтобы суммарное время работы было наименьшим, если:

n	1	2	3
1	12 минут	12 минут	11 минут
2	14 минут	11 минут	8 минут
3	11 минут	13 минут	10 минут

Введите время станка № 1 для детали № 1 : 12

Введите время станка № 1 для детали № 2 : 12

Введите время станка № 1 для детали № 3 : 11

Введите время станка № 2 для детали № 1 : 14

Введите время станка № 2 для детали № 2 : 11

Введите время станка № 2 для детали № 3 : 8

Введите время станка № 3 для детали № 1 : 11

Введите время станка № 3 для детали № 2 : 13

Введите время станка № 3 для детали № 3 : 10

Лучший случай:

Для станка № 1 - деталь № 2

Для станка № 2 - деталь № 3

Для станка № 3 - деталь № 1

Соответственно, получаем, что лучшим решением будет поставить на 1 станок изготавливаться 2 деталь, на 2 станок поставить 3 деталь, а на 3 станке оставить изготавливаться 1 деталь.